Ex No: 06

Date: 10/09/2026

Error Correction at Data Link Layer

AIM:

write a Program to implement error detection and Correction using HAMMING Code Concept. Make a test run to input data stream and verify error correction feature.

Sender.py :-

```
def find_r(k):
    r=0
    while 2**r<k+r+1
        r+=1
    return r
text = input(" Enter text: ")
binary = " "
for ch in text:
    binary += format(ord(ch), "08b")
Print("Binary:", binary)
k = len(binary)
r = find_r(k)
Print("Data bits:", k)
Print("Redundancy bits:", r)
n = k+r
Code = [0]*(n+1)
j=0
for i in range(1, n+1)
    if i &(i-1)!=0
        Code[i] = int(binary[j])
        j+=1
P=1
```

```
while P≤=h:
    Parity = 0
    for i in range (P, h+1):
        if i&P:
            Parity ^= Code[i]
    Code [P] = Parity
    P = P*2
hamming-code = " "
for i in range(1, h+1):
    hamming-code += str (Code[i])
Print (" Hamming Code :", hamming_code)
with open ("Channel. txt", "w") as f:
    f. write (str (k) + "\n")
    f. write (haming_ code)
Print ("Data saved in channel. txt")
```

Receiver. py

~~def binary~~

```
def binary_to_text (binary):
    text = " "
    for i in range (0, len( binary), 8):
        byte = binary [i: i+8]
        if len(byte) = =8:
            text += chr(int (byte, 2))
    return text
with open("channel. txt", "r") as f:
    k = int ( f.readline())
    data = f.readline ().strip()
Print(" Received Hamming Code: ", data)
h = len (data)
code = [0] + [int (x) for x in data ].
```

```
Corrupted_binary = " "
for i in range(1,n+1):
    if i & (i-1)! = 0:
        Corrupted_binary + = str(code[i])
Corrupted_binary = Corrupted_binary[k]
Corrupted_text = binary_to_text(Corrupted_binary)
Print("Corrupted Text:", Corrupted_text)
error = 0
P = 1
While P<=n:
    Parity = 0
    for i in range (P, n+1):
        if i & P:
            Parity ^ = Code[i]
        if Parity:
            error + = P
        P = P*2
if error == 0:
    Print("No Error detected.")
Else:
    Print("Error detected at Position:", error)
    Code[error] = 1-Code[error]
    Print[" Error corrected."]
Corrected_library = " "
for i in range (1, n+1):
    if i & (i-1) ! = 0
        corrected_binary += str(Code[i])
Corrected binary = Corrected_binary[:k]
Corrected_text = binary_to_text(corrected_binary)
Print["Corrected Text:", Corrected_text]
```

OUTPUT :

```
Enter the text : HELLO
Binary data : 0100100001000101010011000100110001001111
Number of data bits : 40
Number of redundant bits : 6
Hamming code : 0101100010000100001010100110001100110001001111
Data saved in channel.txt

Received Hamming Code : ~~010010110111101010010101001100010000011~~01
  ~~00111101001101010100101~~ 01011000100010000 101010011000 1100110000100
                                                               111
Enter error position to simulate (0 for no error) : 0
Corrupted hamming Code : 01011000100001000 010101001100011001100
    001111
No error detected
corrupted binary data : 0100100001000101010011000 100110001001111
Corrupted text : Hello
```

RESULT:

Thus, error detection and correction is done using hamming Code.